

Hallburg

Deine Stimme im Alten Elbtunnel

Faltungshall mit selbst gemessenen Impulsantworten

Abschlussprojekt Programmieren

Worum es geht

- An fünf Orten in Hamburg wurde gemessen, wie der Raum klingt.
- Das Ergebnis je Ort ist eine Impulsantwort: jede Reflexion an jeder Wand.
- Wer ins Mikrofon spricht, hört seine Stimme mit dem Nachhall dieses Ortes.
- Ort auf der Karte wählen, fünf Sekunden sprechen, Hallanteil regeln.
- Das Verfahren heißt Faltungshall und steckt in jedem Hall-Plugin.

Impulsantwort und Faltung

- Eine Impulsantwort ist die Antwort des Raums auf einen einzelnen Impuls.
- Sie enthält den Direktschall und danach jede einzelne Reflexion.
- Die Faltung legt diese Antwort auf jeden Abtastwert der Stimme.
- Direkt gerechnet: $240\,000 \times 66\,000 \approx 10^{10}$ Multiplikationen.

Faltung im Zeitbereich = Multiplikation im Frequenzbereich

→ zweimal FFT, einmal multiplizieren, einmal zurück:
aus 10^{10} Rechnungen werden Millisekunden.

Wie die Impulsantworten entstanden

- Ein Lautsprecher spielt einen exponentiellen Sinus-Sweep, 20 Hz bis 20 kHz.
- Aufgenommen wird zweimal: als Referenz vor dem Lautsprecher und im Raum.
- Die Entfaltung mit dem Inversfilter ergibt die Impulsantwort.
- Sechs Durchgänge je Ort werden gemittelt, das senkt das Rauschen.

Warum kein zerplatzter Ballon?

Der Sweep bringt über 15 Sekunden Energie in den Raum statt nur in einem Moment. Die Verzerrungen des Lautsprechers landen in der Entfaltung vor der Impulsantwort und lassen sich dort abschneiden.

Fünf Orte: Alter Elbtunnel · Parkhaus Landungsbrücken · Passage der Städtepartnerschaften · Unterführung Sternschanze · U-Bahnhof Überseequartier

Das Kernstück: `fft_convolve`

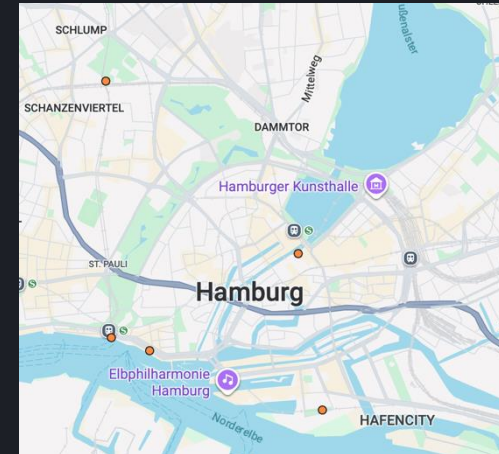
```
def fft_convolve(signal, ir):  
    """Faltet ein Signal mit einer Impulsantwort."""  
    if len(signal) == 0 or len(ir) == 0:  
        return np.zeros(0)  
  
    return fftconvolve(signal, ir)
```

- Beide Signale werden vorher mit Nullen verlängert – sonst faltet die FFT „im Kreis“ und das Ende rutscht in den Anfang.
- Gemessen an einer 5-Sekunden-Aufnahme: rund 6 ms statt Minuten.
- Der Rest von `utils.py`: zuschneiden auf -60 dB und auf Pegel bringen.

Aufbau des Programms

main.py	Start, fängt Fehler beim Programmstart ab
gui.py	StartView · MapView · LocationView
models.py	ImpulseResponse · Location
services.py	Orte einlesen, Aufnahme, Wiedergabe
utils.py	Faltung, Zuschneiden, Skalieren

- Zwei Klassen: ImpulseResponse lädt und bereitet eine WAV auf, Location beschreibt einen Ort.
- Drei Seiten in einem QStackedWidget, ESC führt eine Seite zurück.



Karte mit Markern



Passage der Städtepartnerschaften



Ortsseite mit Aufnahme

Fehlerbehandlung und Tests

try-except steht dort, wo wirklich etwas schiefgehen kann:

locations.json fehlt oder ist kaputt → Meldung beim Start
Eintrag unvollständig → Ort wird übersprungen
WAV fehlt oder ist beschädigt → Ort bleibt sichtbar
Kein Mikrofon oder Gerät belegt → Hinweis statt Absturz
Foto fehlt → if-Abfrage statt except

- 11 Tests, die ohne Aufnahmen laufen.
- Die Faltung wird gegen np.convolve geprüft – ein unabhängiger Vergleich.
- Einheitsimpuls, Stille, leere Eingabe und kaputtes JSON sind abgedeckt.
- Von Hand geprüft: Aufnahme bei 0 %, 70 % und 100 % Hallanteil.

Fazit

Ergebnis

- Fünf Orte, selbst gemessen, vom Sweep bis zur fertigen Impulsantwort.
- Die Faltung läuft in Millisekunden – die Wiedergabe startet sofort.
- Das Fenster bleibt während der Aufnahme bedienbar (QTimer statt Warten).
- Fotos werden beim Einlesen verkleinert: 214 MB gespart.

Gelernt

Warum eine Faltung über die FFT schneller ist als direkt gerechnet.

Dass die Messmethode über das Ergebnis entscheidet. Ein Sweep schlägt den Ballon deutlich.

Wie eine Oberfläche reagierbar bleibt, während sie beschäftigt ist.

Danke fürs Zuhören.